



LITHESPAN

Detailed Venture Plan

Software built for your system.

Evidence-backed releases across languages, runtimes, systems software, packages, container images, and workload-specific machine images.

CORE THESIS

**Define the software, hardware, OS, workload, output, and support term.
Build one traceable release. Attach the evidence. Maintain the promise.**

VERSION 6.0 | 13 AUGUST 2026

lithespan.com | contact@lithespan.com

PLAN MAP

Contents

#	Section
1	Executive summary
2	Venture definition and strategic choice
3	Mission, principles, and public promise
4	Category and positioning
5	Problem and market context
6	Target customers and buying triggers
7	Product architecture
8	Workload-specific machine images
9	Machine-image offer and boundary
10	Release and evidence architecture
11	Formats and distribution
12	Security, compatibility, and benchmarks
13	Services and commercial entry
14	Free and paid boundary
15	Pricing and unit economics
16	Go-to-market
17	Initial wedge and portfolio priorities
18	Roadmap
19	Operating model and metrics
20	Financial posture and risks
21	Brand, website, and messaging
22	First 90 days
23	Strategic decisions
A-E	Product, format, contract, support, and source appendices

SCOPE NOTE

Lithespan builds software releases, not generic cloud infrastructure. Container and machine images are delivery surfaces for tested software. Fleet operation, IAM, networking, and 24x7 SRE remain outside the default offer.

VENTURE RECOMMENDATION

1. Executive summary

Lithespan should become a specialist software engineering and product company for defined operating targets. It can create new software and maintain optimized releases of existing software across languages, compilers, runtimes, systems software, packages, binaries, container images, and machine images.

RECOMMENDED CATEGORY

Evidence-backed software releases for defined systems. The unit of value is a maintained release linked to its target, tests, outputs, and support term.

Decision	Recommendation
Primary promise	Software built for the customer's system, with explicit performance, compatibility, security, and lifecycle evidence.
Commercial entry	Assess one current system, build or migrate one target, then attach recurring maintenance and requalification.
Enterprise image offer	Workload-specific machine images delivered into the customer's account or private repository, with boot, workload, upgrade, rollback, and failure evidence.
Public product	Free public releases and evidence where maintenance capacity and licensing permit.
Differentiation	Cross-layer software knowledge, target definition, benchmark evidence, compatibility history, and lifecycle ownership.
Boundary	Do not sell generic golden-image plumbing or operate customer fleets. Use Packer and cloud-native builders where they fit.

COMPANY DEFINITION

2. Venture definition and strategic choice

Lithespan constructs, validates, distributes, and maintains software for a defined combination of hardware, operating system, workload, output, and support term. The company is broader than infrastructure software and narrower than a general cloud consultancy.

Possible company	Assessment
Artifact catalogue	Easy to distribute, but catalogue breadth and image size are weak moats.
Security distribution	Real demand, but requires a dedicated package and remediation operation and attacks established vendors directly.
Performance consultancy	High-value entry, but it compounds only when work becomes reusable release machinery, evidence, and maintenance.
Generic VM-image practice	Crowded tooling market; risks drifting into cloud operations and compliance implementation.
Evidence-backed release company	Best fit. It spans software creation, optimization, packaging, machine images, migration, and recurring qualification.

CHOSEN PATH

Begin with expert engineering and a small public product surface. Paid work discovers valuable targets. Repeated mechanisms become build, test, evidence, and release capabilities.

OPERATING IDENTITY

3. Mission, principles, and public promise

Mission. Build and maintain software that fits the system it must run on.

Vision. Teams should be able to choose a release by its declared hardware, OS, workload, evidence, output, and support term rather than by opaque defaults or format alone.

Principle	Operational meaning
Evidence before claims	State the baseline, workload, hardware, OS, method, variance, regressions, and trade-offs.
Software breadth	Languages, runtimes, compilers, packages, systems software, and images are all valid products.
Compatibility is a feature	Optimization is incomplete when behavior, upgrade, rollback, or failure boundaries are unknown.
Security is part of the release	Track inputs, SBOMs, provenance, signatures, vulnerabilities, and relevant rebuilds.
Upstream first	Move general fixes upstream when practical and keep owned patch stacks narrow.
Narrow promises	A stable target needs an owner, tests, outputs, upstream watch, and end date.

POSITIONING

4. Category and positioning

The public line is simple: **Software built for your system**. The technical category underneath it is evidence-backed software releases for defined operating targets.

- **Define the target:** software, hardware, OS, workload, format, and support.
- **Build one release:** pinned inputs, traceable output, no rebuild during promotion.
- **Attach evidence:** performance, compatibility, security, upgrade, rollback, and failure results.
- **Project into formats:** packages, binaries, OCI, machine images, archives, and Nix outputs.
- **Maintain the promise:** repeat evidence as upstream software, fleets, and requirements change.

COMMERCIAL POSITION

Cloud vendors and Packer automate image construction. Lithespan defines and qualifies the software system inside the image, proves it against the workload, and maintains the resulting release.

The website should continue to use three top-level pages: What we build, How it works, and Services. Machine images belong inside the product, format, and service narratives until a customer example justifies a dedicated page.

WHY THIS EXISTS

5. Problem and market context

Problem	Mechanism
Broad defaults	Distribution builds, runtimes, and services optimize for general compatibility, not one fleet or workload.
Layered decisions	Kernel, drivers, libraries, runtime, compiler, application, packaging, and image choices are made separately.
Format fixation	Container size or a passing image build says little about CPU, memory, latency, boot, failure, or lifecycle behavior.
Migration risk	Architecture, OS, toolchain, image format, and cloud moves expose dependency and rollback gaps.
Fragmented evidence	Teams struggle to reproduce why a build, configuration, or machine image was approved.
Lifecycle decay	Upstream changes invalidate prior tuning, security, and compatibility assumptions.

Enterprise machine-image demand is validated by mature offerings from AWS, Azure, Google Cloud, and HashiCorp. Their existence confirms the need for standardized, repeatable images. It also makes generic image building a poor point of differentiation. Lithespan must own workload fit and release evidence, not the builder itself.

DEMAND

6. Target customers and buying triggers

Customer	Job to be done
Platform and SaaS teams	Reduce fleet cost or memory use; qualify a new CPU, OS, runtime, or machine image.
Enterprise modernization	Upgrade runtimes and frameworks with explicit behavior and regression evidence.
Regulated enterprises	Produce controlled private images with traceable inputs, policy evidence, rebuild schedules, and customer-account delivery.
Software appliance vendors	Ship a tested system image across cloud, virtualization, and private infrastructure.
Edge and homogeneous fleets	Build for constrained or consistent hardware where target-specific optimization has measurable value.
Software maintainers	Create multi-target binaries, packages, images, provenance, and repeatable qualification.

BEST EARLY BUYER

A platform or product team with a costly stable workload, a meaningful migration or release decision, and enough control over its fleet to act on measured results.

PRODUCT MODEL

7. Product architecture

Capability	Responsibility
Build	Pinned source, dependencies, toolchains, patches, target options, packaging, and repeatability.
Runtime	Languages, compilers, interpreters, standard libraries, JIT/AOT policy, allocators, GC, linking, and startup.
Systems	Proxies, schedulers, storage, messaging, databases, caches, telemetry, native libraries, and operating services.
Images	OCI and machine-image projections that include the agreed OS and software composition.
Verify	Workload, compatibility, security, boot, upgrade, rollback, and failure evidence.
Maintain	Upstream watch, rebuild policy, regression comparison, support window, and end-of-life.

RELEASE EQUATION

Software x hardware x OS x workload x format x support = one defined Lithespan target.

The architecture should remain format-neutral. A native package, OCI image, AMI, VHD, qcow2, and OVA may be distinct artifacts, but all should trace back to the same source, target definition, tests, and release identity.

ENTERPRISE EXTENSION

8. Workload-specific machine images

Machine images are a strong enterprise surface because they bring the operating system, kernel, drivers, runtimes, libraries, services, and application configuration into one testable release boundary.

Segment	Value
Large homogeneous fleets	Faster boot, predictable configuration, repeatable performance, and controlled instance-family qualification.
Regulated environments	Traceable inputs, private builds, security baselines, evidence retention, controlled promotion, and scheduled rebuilds.
Software appliances	A portable product envelope for customer-operated deployments across cloud and virtualization platforms.
Edge and private cloud	Portable qcow2/OVA outputs and hardware-specific software composition.
Architecture or OS migration	One program can qualify the new software stack and its deployment image together.

POSITIONING RULE

Call the offer workload-specific machine images. Avoid custom optimized VMs, which can imply hypervisor engineering, cloud instance design, or managed operations.

COMMERCIAL OFFER

9. Machine-image offer and boundary

Included	Default deliverable
Target definition	Cloud or virtualization platform, base OS, kernel, drivers, software, instance families, regions, workload, security baseline, outputs, and support.
Build	Versioned Packer or provider-native recipe, pinned inputs, private build path, and artifact identifiers.
Qualification	Boot, workload, compatibility, security, upgrade, rollback, failure, and image-distribution tests.
Evidence	SBOM, provenance, checksums, benchmark report, compatibility results, known trade-offs, and support record.
Delivery	AMI, Azure managed image or VHD, Google Cloud custom image, qcow2, or OVA in the customer's account or repository.
Maintenance	Scheduled or event-driven rebuilds, regression comparison, requalification, and end-of-life handling.

BOUNDARY

Lithespan does not default to operating fleets, managing IAM or networks, owning application uptime, or becoming a general cloud migration team. Rollout design and handoff are in scope; ongoing operations require a separate explicit agreement.

BUILD ONCE, PROJECT MANY WAYS

10. Release and evidence architecture

Every release begins with one canonical definition and evidence graph. Promotion changes status and support commitment; it must not rebuild the payload.

```
source + dependencies + target
|-- build and compatibility tests
|-- workload and benchmark suite
|-- security, SBOM, provenance, signatures
|-- boot, upgrade, rollback, and failure tests
`-- immutable release identity
|-- packages / binaries / OCI / archives / Nix
`-- AMI / Azure image or VHD / GCP image / qcow2 / OVA
```

State	Meaning
edge	Experimental output; no stability promise.
candidate	Intended next stable release; qualification in progress.
stable	Maintained target with owner, tests, evidence, outputs, support boundary, and end date.
customer/LTS	Private contract defining versions, platforms, rebuild cadence, response terms, and EOL.

DELIVERY SURFACES

11. Formats and distribution

Format	Initial mechanism	Posture
OCI	GHCR or customer registry	Public standard images; private customer compositions.
Native packages	GitHub Releases first; package indexes later	deb, rpm, apk, pkg, and signed archives from the canonical release.
Nix	Overlay or flake; cache when demanded	First-class reproducibility and dependency closure.
AWS	AMI in customer account; provider mapping in evidence	Private-first workload-specific images.
Azure	Managed image or VHD in customer subscription	Private-first; Compute Gallery when customer architecture requires it.
Google Cloud	Custom image in customer project	Private-first; family and lifecycle mapping.
Portable VM	qcow2 and OVA	Appliance, edge, private cloud, and migration outputs.

The public website should list these formats without implying that every format is available for every release. Each stable target publishes its actual supported outputs.

PROOF

12. Security, compatibility, and benchmarks

Evidence family	Minimum useful record
Performance	Workload, hardware, OS, kernel, duration, warm-up, concurrency, dataset, sample count, variance, baseline, regressions, and trade-offs.
Compatibility	Supported OS, libc, ABI, architecture, dependencies, services, interfaces, upgrades, and known incompatibilities.
Supply chain	Pinned source and dependencies, build identity, SPDX or CycloneDX SBOM, provenance, checksums, signatures, and immutable references.
Machine image	Base image, kernel, drivers, packages, boot path, instance families, metadata settings, network assumptions, storage layout, and distribution mapping.
Lifecycle	Owner, channel, update scope, rebuild trigger, response terms, support window, and end date.

SECURITY POSITION

Security is part of the release and a constraint on optimization. Lithespan should not compete primarily on scanner counts or broad compliance claims until it has the staff and process to own them.

COMMERCIAL ENTRY

13. Services and commercial entry

Service	Deliverables	Commercial form
Assess a system	Reproducible baseline, bottleneck analysis, candidate targets, measured comparison, and recommendation.	Fixed fee
Build or migrate	Tested release for agreed hardware, OS, workload, format, rollout, and rollback.	Project + maintenance
Machine-image program	Image target, recipes, private outputs, boot/workload/security evidence, customer-account distribution, and runbook.	Project + recurring qualification
Maintain and requalify	Upstream watch, rebuilds, repeated evidence, regression reports, and EOL handling.	Annual subscription
Benchmark laboratory	Customer workload harness, version or architecture comparison, and release qualification.	Retainer
Release engineering	Multi-target binaries, packages, OCI, machine images, signatures, SBOM, provenance, and private distribution.	Project or managed pipeline

- 1 Measure the current system.
- 2 Define candidate targets.
- 3 Build traceable releases.
- 4 Test the workload and operating boundaries.
- 5 Select and qualify the release.
- 6 Deliver into the agreed channel.
- 7 Maintain as upstream and fleets change.

ECONOMICS

14. Free and paid boundary

Public work earns trust; revenue starts when Lithespan accepts private complexity, ongoing maintenance burden, or contractual responsibility.

Capability	Posture
Public source, build definitions, and standard releases	Free/open where licensing and maintenance capacity permit
Public benchmarks and compatibility evidence	Free
Common architecture tiers and standard outputs	Free/public
Customer-specific software or dependency composition	Paid/private
Fleet-specific tuning or private workload evidence	Paid/private
Customer-specific machine images	Paid/private
Private repository or customer-account distribution	Paid
Recurring rebuilds and regression qualification	Paid recurring
Lifecycle, compatibility, or response commitments	Paid recurring

WHAT CUSTOMERS PAY FOR

Understanding their system, defining a release target, building the right composition, proving it on relevant infrastructure, delivering it privately, and keeping the promise current.

ECONOMICS

15. Pricing and unit economics

These ranges are hypotheses for customer discovery, not published prices. Price should follow decision value, workload access, platform count, output count, evidence depth, rollout responsibility, and maintenance load.

Offer	Illustrative range	Notes
System assessment	INR 8-25 lakh / US\$15k-50k	One workload and a bounded decision.
Custom software release	INR 20-75 lakh / US\$40k-150k	One or more targets and production qualification.
Machine-image program	INR 25 lakh-1 crore / US\$50k-200k	Platform count, policies, private delivery, testing, and rollout depth drive scope.
Architecture or OS migration	INR 30 lakh-1.5 crore / US\$60k-300k	Native dependencies and rollout ownership dominate.
Benchmark laboratory	INR 3-12 lakh/month / US\$6k-25k/month	Recurring version, target, or image qualification.
Maintenance	INR 15-75 lakh/year / US\$30k-150k/year	Versions, platforms, outputs, rebuild cadence, and response terms.

Every extra OS, cloud, architecture, region, image format, or support promise multiplies requalification work. Contracts should price that matrix explicitly.

MARKET ENTRY

16. Go-to-market

- Publish one compelling end-to-end release: source to package, OCI, machine image, evidence, and support record.
- Use technical comparisons that answer real decisions: OS change, runtime upgrade, architecture move, startup, memory, or workload economics.
- Publish negative results when a smaller image, newer ISA, different allocator, or machine image does not improve the system.
- Offer a fixed-scope assessment as the first purchase; propose build and maintenance only after the baseline supports it.
- Use machine images to enter enterprise platform, appliance, regulated, and private-cloud conversations without becoming a cloud-operations vendor.
- Partner with cloud, platform, performance, migration, and security specialists where responsibilities remain explicit.

Channel	Use
lithespan.com	Clear category, products, release model, services, evidence, and contact.
GitHub	Public source, releases, issues, evidence, and reference artifacts.
Technical publication	Benchmark methods, architecture analysis, case studies, and negative results.
Founder network	Principal engineers, platform leaders, and product vendors with known release decisions.
Partners	Cloud migration, appliance, regulated infrastructure, and secure substrate opportunities.

FOCUS

17. Initial wedge and portfolio priorities

Target	Year-1 posture	Reason
Language/runtime flagship	Production-supported	Choose one area with founder depth and measurable compiler, runtime, startup, memory, or compatibility work.
Systems-software flagship	Production-supported	Choose one proxy, telemetry, data, or native system with clear workload contracts.
Packaging and evidence	Production-supported	Prove packages, OCI, SBOM, provenance, benchmarks, and support state share one release identity.
Machine-image design partner	Private paid pilot	Qualify one AWS, Azure, GCP, or portable image for a real workload and customer account.
Broader catalogue	Demand-gated	Avoid publishing versions, platforms, and formats that cannot be maintained.

FOCUS CORRECTION

Do not make the first catalogue synonymous with infrastructure software. The venture must visibly support language implementations, compilers, runtimes, and other software where target-specific engineering creates value.

ROADMAP

18. Roadmap

0-12 months

- Publish the release contract, evidence schema, support channels, and one complete public release.
- Sell the first assessment and one build or migration engagement.
- Run one paid machine-image design partner with private customer-account delivery.
- Measure maintenance hours before adding platforms or formats.

12-36 months

- Expand only the product families and formats with recurring demand and owned tests.
- Productize private build, evidence, image promotion, and rebuild workflows.
- Add package indexes, caches, and controlled distribution when customers require them.
- Build reusable cloud mappings for AWS, Azure, and Google Cloud without hiding provider semantics.

3-7 years

- Maintain a longitudinal performance and compatibility graph across software versions, platforms, and workloads.
- Automate candidate selection and release qualification while keeping target ownership explicit.
- Offer a private release control plane for policy, build, evidence, promotion, rollback, and lifecycle.

EXECUTION

19. Operating model and metrics

Role	Primary responsibility
Founder / principal engineer	Category, architecture, flagship releases, benchmark credibility, customer discovery, and major engagements.
Build and release engineer	Multi-platform CI, packaging, OCI, machine images, provenance, signing, and distribution.
Runtime / compiler engineer	Languages, toolchains, JIT/AOT, GC, allocators, linking, startup, and architecture behavior.
Systems / platform engineer	OS, kernel, drivers, storage, networking, native libraries, VM formats, and cloud mappings.
Benchmark / reliability engineer	Workload harnesses, statistics, boot, upgrade, rollback, failure, and regression testing.
Solutions engineer	Assessment, target definition, private delivery, rollout handoff, and maintenance scope.

Decision metrics

Evidence completeness; assessment-to-build conversion; maintenance attachment; verified customer outcome; engineer-hours per stable target; release failure rate; recurring revenue; renewal; and the fraction of paid work that becomes reusable machinery.

ECONOMICS AND CONTROL

20. Financial posture and risks

Lithespan should remain capital-light. Public distribution and website delivery are cheap; expert engineering, customer test infrastructure, cloud image storage, and the lifecycle matrix are the real costs. Customer-specific benchmark and image-build spend should be passed through or priced into the engagement.

Risk	Control
Catalogue and format sprawl	Demand gates, stable/candidate/edge channels, explicit target owner, and support matrix.
Machine-image drift into cloud operations	Written scope boundary, customer-account delivery, handoff runbook, and separate operations agreement if ever needed.
Cloud-builder commoditization	Use existing builders; differentiate on target engineering, workload evidence, compatibility, and maintenance.
Security or compliance overclaim	Narrow claims, explicit evidence, external specialists where needed, and no blanket compliance guarantees.
Services trap	Fixed targets, reusable build/test/evidence output, maintenance contracts, and refusal of open-ended staff augmentation.
Benchmark gaming	Open methods, context, variance, trade-offs, negative results, and customer reproduction where possible.
Upstream divergence	Upstream general fixes, own every patch, and retire targets that cannot be maintained.

BRAND

21. Brand, website, and messaging

Surface	Decision
Tagline	Software built for your system.
Descriptor	Languages, compilers, runtimes, systems software, packages, binaries, container images, and machine images for defined targets.
Proof line	Each release includes performance, compatibility, security, and lifecycle evidence.
Navigation	What we build / How it works / Services. Security remains part of How it works, not a fourth top-level product.
Primary CTA	Discuss a target.
Machine-image language	Workload-specific machine images. Avoid custom optimized VMs.
Scope line	Public releases are free. Private builds, migration, recurring qualification, and guaranteed support are paid.

HIGH-SNR RULE
Public copy states what Lithespan builds, what defines a target, what evidence comes with a release, and what customers pay for. Internal architecture language belongs in the plan and technical documentation.

EXECUTION PLAN

22. First 90 days

Window	Outcome
Weeks 1-2	Publish the target and evidence contract; define stable/candidate/edge; prepare one flagship software target.
Weeks 3-4	Build the canonical source-to-release path with package, OCI, evidence, and signing outputs.
Weeks 5-6	Ship the first public release and benchmark report; measure maintenance effort.
Weeks 7-8	Create the machine-image reference path using Packer or one provider-native builder; include boot and rollback tests.
Weeks 9-10	Run enterprise discovery with platform, appliance, regulated, and migration buyers; price one bounded pilot.
Weeks 11-12	Execute one assessment or design-partner engagement; deliver privately into the customer account.
Week 13	Review evidence quality, customer outcome, conversion, maintenance hours, and whether the image offer stays within scope.

90-DAY GATE

Do not create a broad image catalogue. One real customer target with repeatable evidence is more valuable than many unmaintained AMIs or VM files.

DECISION

23. Strategic decisions

- 1 Lithespan builds all kinds of software where a defined operating target creates value; it is not an infrastructure-software-only company.
- 2 The public promise is Software built for your system.
- 3 A target is defined by software, hardware, OS, workload, format, and support.
- 4 Performance, compatibility, security, and lifecycle evidence are part of the release.
- 5 Container and machine images are delivery surfaces, not the company category.
- 6 Workload-specific machine images are a credible enterprise offer when they include private delivery, workload qualification, and recurring rebuilds.
- 7 Cloud and Packer builders should be used rather than rebuilt; Lithespan differentiates on the software system and evidence.
- 8 Fleet operations, IAM, networking, and general cloud migration remain outside the default offer.
- 9 Public standard releases remain free where they can be maintained; private composition, migration, requalification, and guarantees are paid.
- 10 Expansion is controlled by maintenance capacity and customer evidence, not buildability or catalogue size.

ONE-SENTENCE PLAN

Use free, evidence-backed software releases to earn trust; sell customer-specific target definition, composition, proof, private delivery, maintenance, and accountability.

APPENDIX

A. Product and customer examples

Product family	Example target
Language or compiler	A compiler, standard library, interpreter, or toolchain built for an OS, architecture, workload, and distribution model.
Runtime	Python, JVM, JavaScript, .NET, Ruby, Go, Rust, or another runtime qualified for startup, memory, throughput, compatibility, or fleet constraints.
Systems software	Proxy, scheduler, telemetry collector, database, cache, messaging system, storage service, or native library.
Native package	A signed deb, rpm, apk, pkg, Nix output, binary, or archive with explicit ABI and dependency assumptions.
Container image	OCI output linked by digest to the same tested payload and evidence as other formats.
Machine image	AMI, Azure image or VHD, GCP image, qcow2, or OVA containing the agreed OS and software composition.

Candidate early engagements should be selected by measurable decision value, founder expertise, and the ability to create reusable release machinery. The plan deliberately avoids naming one permanent first product before customer discovery.

APPENDIX

B. Output and platform matrix

Output	Target-specific fields	Default posture
Native binary/package	OS, distro, libc, ABI, architecture, dependency set, signature, repository.	Public standard or private
OCI image	Base, user, entrypoint, filesystem, runtime, architecture, registry, digest.	Public standard or private
AWS AMI	Base AMI, instance families, regions, boot mode, drivers, storage, sharing, lifecycle.	Private customer account
Azure image/VHD	Source, generation, security type, VM sizes, gallery, regions, subscription, lifecycle.	Private customer subscription
GCP custom image	Source, machine families, guest environment, project, family, storage location, lifecycle.	Private customer project
qcow2/OVA	Firmware, virtual hardware, drivers, storage, cloud-init or metadata, import targets.	Private/appliance

Architecture should be described in terms of the actual target and required capabilities. Public copy should not elevate any one ISA generation into the company identity.

APPENDIX

C. Machine-image release contract

A machine-image release extends the general target contract with image-specific fields:

```
release_id: customer-product-2026.08.1
software: source, version, dependencies, configuration
platform: provider, account/project, region, image format
os: distribution, version, kernel, libc, services
hardware: architecture, instance families, required features
image: base identity, boot mode, drivers, storage, metadata
workload: harness, baseline, success thresholds
evidence: boot, performance, compatibility, security, upgrade, rollback, failure
distribution: private destination, regions, sharing policy
support: owner, rebuild triggers, response terms, end date
```

PROMOTION RULE

A stable image must identify the exact artifact already tested as candidate. Rebuilding during promotion invalidates the evidence chain.

APPENDIX

D. Support and benchmark standards

- Stable targets require an owner, upstream watch, compatibility suite, benchmark or workload evidence, build pipeline, output list, and support date.
- Machine images add boot, upgrade, rollback, failure, provider import, distribution, and instance-family qualification.
- Public issue support is best effort. Paid obligations exist only in written agreements.
- Relevant upstream security fixes trigger impact review, rebuild, and repeated qualification for maintained targets.
- Negative results and regressions remain visible; performance gains do not erase compatibility or operating trade-offs.
- Customer traces, dependencies, configurations, identities, fleet data, and private images remain private.

Metric	Required context
Performance	Baseline, workload, hardware, OS, duration, warm-up, concurrency, data, sample count, variance, and trade-offs.
Boot/startup	Image identity, instance or hypervisor, firmware/boot mode, storage, network readiness, service readiness, and retries.
Compatibility	Interfaces, dependencies, ABI, services, data formats, upgrades, rollback, and known failures.
Lifecycle	Upstream version, rebuild trigger, release cadence, response term, support owner, and EOL.

APPENDIX

E. Research sources and assumptions

- HashiCorp Packer documentation - identical machine images for multiple platforms from one source configuration: <https://developer.hashicorp.com/packer/docs>
- AWS EC2 Image Builder documentation - automated build, test, management, and distribution of AMIs and container images: <https://docs.aws.amazon.com/imagebuilder/>
- Azure VM Image Builder overview - image customization, private builds, Compute Gallery, managed images, and VHD outputs: <https://learn.microsoft.com/en-us/azure/virtual-machines/image-builder-overview>
- Google Cloud Compute Engine custom images - reusable custom images, image families, projects, storage locations, and encryption choices: <https://cloud.google.com/compute/docs/images/create-custom>
- Open Container Initiative specifications: <https://opencontainers.org/>
- Nix documentation for reproducible packages and binary caches: <https://nix.dev/>
- GitHub documentation for Releases, Packages, Container Registry, and Pages: <https://docs.github.com/>
- SPDX specification: <https://spdx.dev/> and CycloneDX specification: <https://cyclonedx.org/>
- SLSA supply-chain framework: <https://slsa.dev/>

This document combines strategy with technical and commercial hypotheses. Provider capabilities, pricing, support policies, compliance mappings, and licensing obligations can change and must be revalidated before implementation or contract. The document is not legal, accounting, security, or compliance advice.